

High Level Design (HLD) : AI-Based Pond Planning

Page _____

Date _____

Problem Statement & Objectives

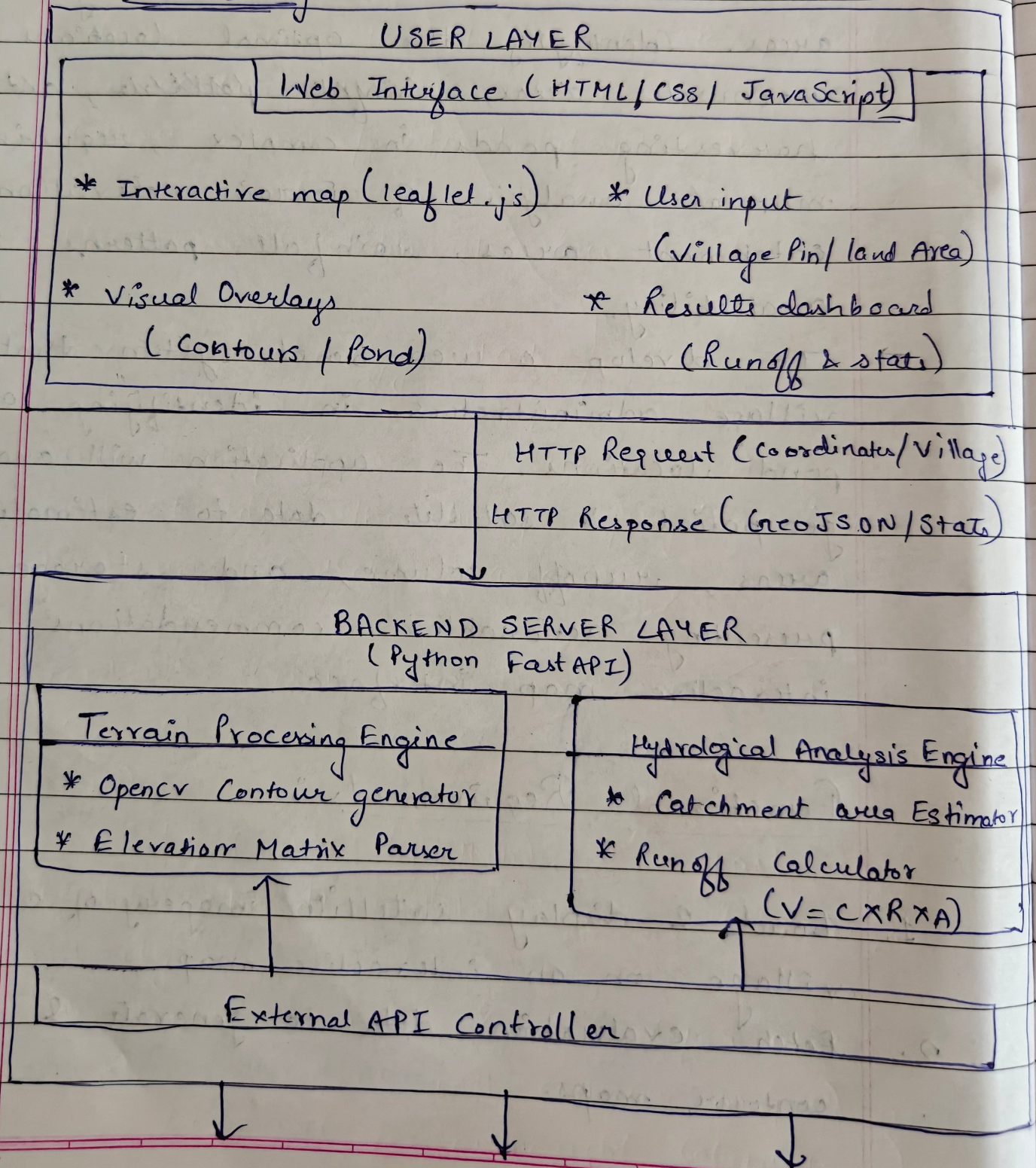
- Water conservation is a serious challenge in rural areas. Identifying the optimal locations for rainwater areas, rainwater patterns, and harvesting ponds is complex, requiring the manual analysis of terrain elevation, catchment areas, rainfall patterns & land availability.
- To develop a web-based system that assists village administrators in identifying optimal pond locations. The application will analyze geospatial & satellite data to estimate catchment areas, runoff volume, and storage capacity, presenting actionable recommendations on an interactive map interface.

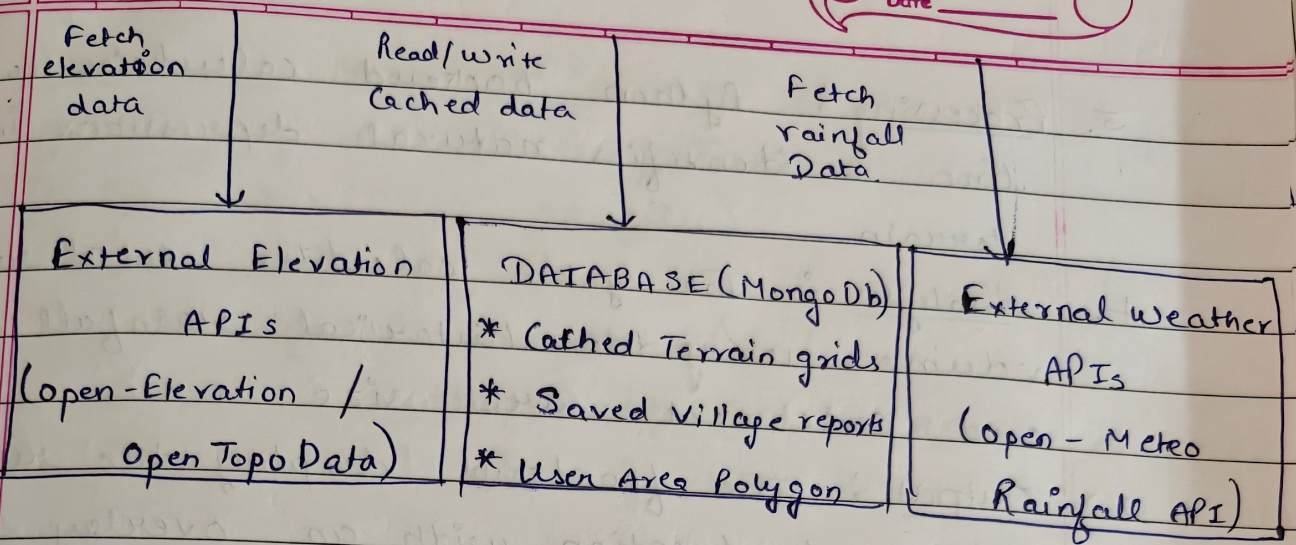
Functional Requirements :

1. Load & display satellite imagery of a selected village on an interactive map.
2. Fetch evaluation data to generate & visualize contour maps.

3. Identify low-evaluation zones suitable for pond excavation.

Block Diagram





* Function requirements continue :-

4. Estimate the catchment area that will drain into the selected pond location.
5. Query historical rainfall data using public APIs.
6. Calculate expected water runoff volume based on rainfall & catchment area.
7. Recommend pond dimensions (depth & capacity).
8. Overlay all metrics, coordinates & recommendation on the map UI.

Project workflow

1. Input: User selects searches for a village or drops a pin on the map.
2. Data Acquisition: System calls elevation & Rainfall APIs for the selected coordinates.

3. Processing: Python backend calculates contour lines & identifies natural depressions in the terrain.
4. Analysis: The system calculates total runoff volume using the historical rainfall data and catchment area size.
5. Output: The frontend displays the recommended pond location with an overlay showing dimensions, catchment area & stats.

Proposed Technology Stack

Since the focus is on Science & AI, the backend handles the heavy lifting while the frontend is kept basic.

- Frontend: HTML, CSS, JavaScript & Leaflet.js (a simple, open-source library for interactive maps).
- Backend: FastAPI (Python). It is fast, easy to build & handles data science libraries natively.
- Data Processing / AI: Opencv (for processing elevation matrices into contour maps) and Numpy / Pandas (for numerical calculations).

- Database : MongoDB (NoSQL is excellent for storing unstructured JSON data like coordinates & village profiles).
- External APIs :
 - Rainfall : Open-Metro API (free, no API key required for historical weather).
 - Elevation : Open-Elevation API or OpenTopoData

API Design

These are the core backend routes the frontend will communicate with :

- GET / api / location / {village-name}
 - Purpose : Fetches the bounding box coordinates & basic info for a given village.
- POST / api / terrain / contours
 - Payload : { "lat" : 21.1, "lon" : 81.3, "radius" : 5 }
 - Purpose : Fetches elevation data & return processed contour map data (GeoJSON format)
- GET / api / weather / rainfall
 - Parameters : lat, lon
 - Purpose : Retrieves the last 10 yrs of average of annual rainfall.
- POST / api / pond / recommendation
 - Payload : { "catchment-area-sqm" : 5000, "rainfall-mm" : 12000 }

→ Purpose: Returns the estimated runoff volume, recommended depth, storage capacity.

Algorithms / Methodology to be used

1. Terrain Analysis (Contour Mapping):
 - Fetch a grid of exact elevation points (Digital Elevation Model).
 - Use OpenCV to treat the elevation grid as an image matrix.
 - Apply edge detection & contouring algorithms (eg. CV2.findContours) to highlight area with the lowest relative elevation.
2. Catchment Area Estimation:
 - Identify the lowest point (sink).
 - Use a basic "Water Drop" or gradient concept: analyze neighboring elevation grids to see which surrounding areas slope downwards toward the selected sink.
3. Runoff volume Calculation:
 - Use the standard Rational Equation for hydrology:
$$V = C \times R \times A$$
 - where V = Runoff volume
 - C = cutoff Coefficient
 - R = Annual rainfall
 - A = Catchment area

4. Pond Dimension Recommendation:

- Based on V , Calculate depth assuming a standard pond geometry (eg. inverted truncated pyramid).

Expected Challenges & proposed Solutions

◦ Challenge 1: Slow external API responses

- Fetching large grids of elevation data can take time & cause the web app to freeze.
- Proposed Solution: Implement database caching. Once elevation data for a specific village is fetched, save it in MongoDB so future queries for the same village load instantly.

◦ Challenge 2: Identifying "Government Land"

- Real-time land ownership records are usually not available via free public APIs.
- Proposed solution: for this assignment, we will simulate this by allowing the user to draw a "polygon" on the map representing available land, & the algorithm will only search for optimal locations within that specific polygon.

◦ Challenge 3^o: Complex GIS Processing

- standard GIS software (like QGIS) is heavy & hard to integrate into a simple web app.
- Proposed solution: Treat elevation grids strictly as 2D Numpy arrays & use OpenCV for mathematical image processing, which keeps the backend lightweight & leverages DSAI skills.